

HERMES STACK

Informe Técnico de Arquitectura Integrada

Versión Definitiva v3.0

Síntesis de Arquitectura General, Agentes, Pipelines, Core,
Visión de Futuro y Justificación de Decisiones

Autor: Erik José Hernández
Email: erikjosehernandez@gmail.com
Repositorio: github.com/erikjosehrndz-crypto/hermes-stack
Entorno: el80.space
Fecha: Mayo 2026

Plataforma de Inteligencia Artificial Conversacional por Voz Autohospedada.
Documento consolidado de ingeniería y especificaciones de baja latencia.

Índice general

1. Sistema General	1
1.1. Topología de Redes Docker	1
1.2. Hardening de Seguridad y Aislamiento Perimetral	1
1.3. Subdominios y Direccionamiento	2
2. Agents	3
2.1. Gestión de Memoria y Contexto Conversacional	3
2.2. Checkpointing y Mitigación de Pérdida de Estado	3
2.3. Orquestación Jerárquica de Modelos	4
3. Pipelines	5
3.1. Pipeline de Voz End-to-End	5
3.2. Pipeline CI/CD (GitHub Actions a VPS)	6
3.3. Pipeline de Observabilidad y Auto-Curación	6
4. Core	7
4.1. LiteLLM Router	7
4.2. Redis Cache	7
4.3. Caddy Proxy	8
5. Visión para el Futuro	9
5.1. Frontend y Next.js Compatibility	9
5.2. Blueprint de Aplicación Android Nativa	9
5.3. Hitos del Roadmap Técnico (2026 - 2027)	10
6. Meta	11
6.1. Estructura de Reglas de Agentes y Sistema	11
6.2. Bucle Operativo de Sesión	11
6.3. El Comando Evolve	12
7. Justificación	13
7.1. STT Local con Whisper en Infraestructura Propia	13
7.2. ElevenLabs Flash v2.5 con auto_mode y Compresión Nula	13
7.3. LiteLLM sobre Integración Propietaria de APIs	14
7.4. Redundancia Híbrida: OpenRouter y API Directa de Google	14

Índice de cuadros

1.1. Tabla de Direccionamiento de Subdominios	2
---	---

Capítulo 1

Sistema General

El **Hermes Stack** es una infraestructura autohospedada basada en microservicios dockerizados, desplegada en un único servidor VPS bajo el dominio primario el80.space. Su principal objetivo es coordinar el procesamiento en tiempo real de voz a voz con baja latencia y alta privacidad.

1.1 Topología de Redes Docker

Para aislar el tráfico de datos funcionales del tráfico de monitorización y observabilidad, se definen dos redes puente (bridge) separadas en el orquestador:

1. **backend**: Red privada para los servicios que integran el flujo conversacional. El agente principal, el reconocedor Whisper, el router LiteLLM y el caché Redis conviven en este segmento sin visibilidad directa hacia la red externa.
2. **monitoring**: Red específica de observabilidad. Conecta Prometheus con Grafana y los distintos exportadores de hardware/kernel (cAdvisor y node-exporter) para aislar la telemetría del núcleo operativo del sistema.

1.2 Hardening de Seguridad y Aislamiento Perimetral

La seguridad perimetral se basa en el principio de defensa en profundidad:

- **Exposición a Loopback**: Todos los puertos definidos en `docker-compose.yml` (excepto el proxy inverso Caddy) están vinculados explícitamente a `127.0.0.1`. Esto anula la accesibilidad por IP pública directa y obliga a que todo el tráfico entrante sea evaluado por las reglas de routing y TLS de Caddy.

- **Usuarios No-Root:** Servicios clave como LiteLLM y Grafana ejecutan procesos bajo identidades de sistema con ID de usuario restringidos (user: "472:472"), evitando la escalada de privilegios en el host ante una potencial vulnerabilidad de ejecución remota de código (RCE).
- **Banderas de Seguridad:** Se aplican restricciones en el kernel del contenedor mediante security_opt con la opción no-new-privileges:true y se descartan las capacidades del sistema con cap_drop: [ALL] para Redis y Hermes.
- **Filesystem de Solo Lectura y tmpfs:** Para evitar la inyección persistente de malware, los volúmenes operativos se montan en modo de solo lectura (:ro) cuando es viable, mientras que las escrituras de archivos temporales se realizan sobre memoria RAM utilizando configuraciones tmpfs (/app/tmp:size=64M,noexec,nosuid).

1.3 Subdominios y Direccionamiento

El proxy inverso **Caddy** actúa como el punto de terminación TLS y enrutador HTTPS. La siguiente tabla describe el mapa de subdominios configurado en producción:

Subdominio	Puerto Interno	Propósito Principal
hermes.el80.space	:8080	Entrada del canal de voz del Agente Hermes (/process)
docs.el80.space	:3001	Frontend de documentación y control (Next.js 14)
litellm.el80.space	:4000	Endpoint unificado del proxy de modelos de lenguaje
grafana.el80.space	:3000	Acceso web a métricas y visualización de Dashboards
files.el80.space	:8095	Gestor web de archivos locales (FileBrowser)

Cuadro 1.1: Tabla de Direccionamiento de Subdominios

Capítulo 2

Agents

El componente **hermes-agent** es el núcleo inteligente y conversacional de la plataforma. Está construido sobre Python 3.12 usando programación asíncrona (`aiohttp`) para responder a peticiones concurrentes de audio y texto de forma ágil.

2.1 Gestión de Memoria y Contexto Conversacional

Para ofrecer interacciones coherentes en el tiempo, el agente no debe ser sin estado (*stateless*). Sin embargo, mantener el histórico en la memoria del proceso genera riesgos de fugas de memoria (*memory leaks*) y pérdida de contexto ante reinicios.

- El agente utiliza **Redis** como capa de estado centralizada.
- Cada sesión de chat se asocia a un identificador único de conversación.
- El contexto, compuesto por el historial de mensajes formateado y variables de sesión, se persiste y recupera en Redis con políticas de expiración (TTL) automáticas de 30 minutos.

2.2 Checkpointing y Mitigación de Pérdida de Estado

Para resolver vulnerabilidades asociadas a la pérdida de estado en la ejecución del agente, se implementa un mecanismo de **checkpointing**:

Flujo de Checkpointing en Hermes

Antes de enviar el prompt final al router LLM, el estado actual de la conversación y las variables del agente se serializan y escriben transaccionalmente en Redis. Si el proceso de inferencia de voz o

el servidor de ElevenLabs falla a mitad del stream, el cliente puede reconectarse a un nodo redundante y reanudar la interacción usando el checkpoint almacenado en Redis, evitando la repetición del turno conversacional del usuario.

2.3 Orquestación Jerárquica de Modelos

La arquitectura define una jerarquía operativa multi-modelo basada en la economía de tokens y la capacidad cognitiva requerida por la tarea:

1. **Orquestación General y Razonamiento Crítico:** Se delega en modelos Premium como gpt-4o o claude-sonnet-4.6 (vía OpenRouter) debido a su alto rendimiento en la toma de decisiones lógicas de negocio.
2. **Extracción de Datos y Tareas Secundarias:** Reservado para modelos intermedios como llama-3.1-70b-instruct.
3. **Inferencia de Conversación Rápida:** Utiliza de forma nativa gemini-2.0-flash gracias a su latencia extremadamente baja de inferencia, óptima para flujos de voz continua.

Capítulo 3

Pipelines

Los flujos de ejecución en el Hermes Stack se dividen en tres pipelines automatizados principales: Voz E2E, Despliegue CI/CD y Observabilidad/Autocuración.

3.1 Pipeline de Voz End-to-End

El flujo que procesa la interacción por voz del usuario de extremo a extremo está optimizado para cumplir con un SLO de **<500 ms**:

1. **Captura y Stream:** El micrófono del cliente envía el audio comprimido a través de un WebSocket seguro (`wss://hermes.el80.space/process`). :443 de Caddy recibe la petición y la deriva localmente al :8080 del agente Hermes.
2. **Speech-to-Text (STT):** El agente envía los bytes del audio al microservicio local de Whisper (:9000). La transcripción se realiza localmente utilizando el modelo base en menos de 200 ms.
3. **Orquestación e Inferencia LLM:** Con el texto transcrito, el agente consulta a :4000 (LiteLLM Router). Si la respuesta no está cacheada en Redis, LiteLLM enruta la petición al proveedor óptimo según la estrategia de latencia, retornando el texto en aproximadamente 300 ms.
4. **Text-to-Speech (TTS):** El agente procesa y limpia el texto para evitar que el sintetizador vocal intente leer caracteres Markdown. El texto limpio se envía mediante WebSocket a la API externa de ElevenLabs Flash v2.5.
5. **Inferencia Vocal y Reproducción:** ElevenLabs, configurado con `auto_mode: true`, devuelve inmediatamente los fragmentos de audio PCM a 24 kHz que se reproducen en streaming en el altavoz del usuario.

3.2 Pipeline CI/CD (GitHub Actions a VPS)

El despliegue continuo garantiza cambios de código ágiles sin detener la operación de producción:

- **Etapas de Lint y Validación:** Todo push o PR ejecuta validaciones con ruff, black y mypy en un runner externo de GitHub.
- **Despliegue Automático:** Al hacer push sobre la rama main, el runner establece conexión SSH con el VPS, descarga los cambios e inicia el build de contenedores (docker compose up -d --build).
- **Health Checks Autenticados:** El pipeline espera un período de gracia de 15 segundos y evalúa la salud de los servicios críticos (LiteLLM y Hermes Agent) usando headers de autenticación (LITELLM_MASTER_KEY).
- **Rollback a HEAD~1:** Si cualquiera de los health checks retorna un estado fallido, el script aborta inmediatamente y ejecuta git checkout HEAD~1 && docker compose up -d --build para restablecer la versión estable anterior, mitigando cualquier downtime de producción.

3.3 Pipeline de Observabilidad y Auto-Curación

Para proteger la disponibilidad del stack (99.5 % de SLA objetivo):

1. **Scraping de Métricas:** Prometheus monitoriza cada 15 segundos los endpoints de telemetría de LiteLLM, Hermes Agent y exporters de sistema (cAdvisor y node-exporter).
2. **Bucle Reactivo de Auto-Curación:** Si un contenedor deja de responder a sus health checks internos de Docker, el daemon autoheal lo detecta a través del socket de Docker y ejecuta un reinicio forzado del servicio afectado en menos de 30 segundos.
3. **Watchdog Systemd del Host:** Un script persistente monitoriza el estado global de Docker. Si el servicio de Docker se congela a nivel del kernel del VPS, envía una alerta al webhook de Slack de emergencias y reinicia el servicio de systemd (systemctl restart docker) de forma automática.

Capítulo 4

Core

El núcleo de procesamiento se compone de tecnologías clave destinadas a centralizar la lógica de integración, enrutamiento y caché.

4.1 LiteLLM Router

Actúa como gateway y proxy unificado para todos los modelos lógicos de lenguaje. En lugar de forzar al agente a implementar clientes específicos de API para cada proveedor, LiteLLM expone una interfaz OpenAI-compatible en el puerto :4000.

- Carga dinámicamente las credenciales seguras de `OPENROUTER_API_KEY` y `GEMINI_API_KEY`.
- Ejecuta un balanceo inteligente basado en la latencia histórica de las peticiones.
- Soporta virtual keys para controlar de forma granular los presupuestos de consumo de tokens.

4.2 Redis Cache

Redis actúa como intermediario para interceptar peticiones de inferencia duplicadas. Si el hash del prompt enviado coincide con un registro activo en Redis, la respuesta se devuelve directamente al agente sin realizar llamadas externas a las APIs de OpenRouter o Google, logrando:

- Latencia de respuesta de **0 ms**.
- Ahorro directo en costos de infraestructura y tokens consumidos.
- Resiliencia del sistema ante cortes prolongados de la red externa o caídas globales de proveedores.

4.3 Caddy Proxy

Es el único componente que escucha en los puertos externos expuestos del VPS (:80 y :443). Maneja la encriptación mediante certificados automáticos SSL y distribuye las peticiones entrantes usando hosts virtuales hacia los puertos de loopback de los contenedores Docker en el backend.

Capítulo 5

Visión para el Futuro

El roadmap de evolución del Hermes Stack plantea mejoras sustanciales en portabilidad, experiencia de usuario y arquitectura.

5.1 Frontend y Next.js Compatibility

El sitio web principal (docs.el80.space) se basa actualmente en Next.js con App Router.

- **Objetivo a corto plazo:** Consolidar la rama `feat/nextjs-rocket-compat` en la rama principal.
- **Impacto:** Permitirá la integración y renderizado ágil de los componentes estáticos y dinámicos del dashboard, facilitando una experiencia de observabilidad en tiempo real desde el navegador del usuario.

5.2 Blueprint de Aplicación Android Nativa

Para masificar el acceso al asistente, se ha diseñado un blueprint arquitectónico de una aplicación móvil nativa:

- **Stack Móvil:** Kotlin con Jetpack Compose para interfaces rápidas y reactivas, junto a la librería de integración de Google AI Studio.
- **Canal WebSocket Nativo:** La app se conectará por WebSockets directos al endpoint de audio del Hermes Agent, permitiendo un canal de comunicación de flujo bidireccional y eliminando por completo la necesidad de un navegador web móvil.
- **Validación Local de Salud:** El cliente móvil consultará periódicamente a `docs.el80.space/health` para verificar el estado de los servicios internos antes de iniciar transmisiones de voz, notificando al usuario si el servidor requiere mantenimiento.

5.3 Hitos del Roadmap Técnico (2026 - 2027)

- **Q3 2026 (Corto Plazo):** Completar el build-check y resolver la latencia en transacciones Whisper mediante streaming de entrada.
- **Q4 2026 (Mediano Plazo):** Migrar los secretos almacenados en texto plano en el archivo `.env` del VPS hacia soluciones de bóveda segura como HashiCorp Vault.
- **2027 (Largo Plazo):** Implementar despliegues avanzados tipo Blue-Green para eliminar cualquier micro-downtime residual durante reinicios de contenedores.

Capítulo 6

Meta

La gestión interna y las normas operativas que rigen el mantenimiento autónomo del Hermes Stack se definen en esta sección Meta.

6.1 Estructura de Reglas de Agentes y Sistema

Para asegurar que los agentes autónomos de IA no rompan la estabilidad operativa de producción, se implementan archivos de control rígidos:

- **[CLAUDE.md](file:///root/CLAUDE.md):** Define el protocolo obligatorio de inicio y cierre de sesión de trabajo, el uso del comando `make check` antes de commits y la configuración del token HTTPS para git push.
- **[AGENTS.md](file:///root/AGENTS.md):** Detalla la secuencia de arranque obligatoria, las restricciones estrictas de Git (como no commitar bases de datos ni dotfiles) y los criterios formales para la definición de una tarea como completada (Definition of Done).

6.2 Bucle Operativo de Sesión

Cada ciclo de trabajo para un agente de desarrollo cumple un protocolo estricto:

1. **Clock-In:** Diagnóstico inicial del stack y backlog mediante comandos locales del sistema.
2. **Tarea Única:** Para evitar la dispersión de esfuerzos, se prohíbe abrir más de una tarea operativa simultáneamente en una sesión de desarrollo.
3. **Cierre y Clock-Out:** Actualización del estado en `PENDIENTES.md`, commits incrementales y generación de un archivo de handoff si la tarea no se culminó.

6.3 El Comando Evolve

El comando `/evolve` es un mecanismo meta-cognitivo para los agentes. Permite analizar la sesión de trabajo recién terminada, detectar puntos débiles en las instrucciones y actualizar automáticamente los archivos de directivas de desarrollo (`CLAUDE.md` y `AGENTS.md`) para que los agentes futuros no vuelvan a tropezar con problemas ya resueltos.

Capítulo 7

Justificación

Las decisiones de diseño de la arquitectura del Hermes Stack han sido seleccionadas estratégicamente para optimizar los recursos, asegurar la disponibilidad y cumplir con las expectativas de rendimiento.

7.1 STT Local con Whisper en Infraestructura Propia

- **Decisión:** Ejecutar el modelo Whisper Base en local dentro de Docker en lugar de APIs externas como OpenAI.
- **Justificación:** El procesamiento local elimina la latencia de subir audio a través de internet (round-trip de red de subida). Además, garantiza que las conversaciones por voz de los usuarios nunca salgan del servidor de producción, cumpliendo con estrictas directrices de soberanía de datos y privacidad.

7.2 ElevenLabs Flash v2.5 con auto_mode y Compresión Nula

- **Decisión:** Configurar la síntesis de voz mediante ElevenLabs Flash v2.5, configurando `auto_mode: true` y `compression=None`.
- **Justificación:** Flash v2.5 reduce la latencia de inferencia TTS a la mitad en comparación con Turbo v2.5 (75ms vs 150ms). El parámetro `auto_mode` deshabilita buffers internos para generar audio PCM inmediatamente en cuanto el agente envía el texto, mientras que omitir la compresión web reduce los ciclos de procesamiento de CPU requeridos para deserializar datos.

7.3 LiteLLM sobre Integración Propietaria de APIs

- **Decisión:** Utilizar LiteLLM Router en lugar de llamadas directas a las APIs de OpenRouter y Google.
- **Justificación:** LiteLLM proporciona una capa unificada de fallback y balanceo basada en latencia de manera nativa sin sobrecargar el código de control del agente. La configuración externa de modelos en YAML facilita cambios inmediatos de proveedores en caliente sin alterar la lógica de negocio de Hermes.

7.4 Redundancia Híbrida: OpenRouter y API Directa de Google

- **Decisión:** Emplear de forma paralela la pasarela de OpenRouter y la API nativa de Google Gemini.
- **Justificación:** Esta topología garantiza alta disponibilidad. Si los servidores de OpenRouter sufren latencias altas o caídas, LiteLLM redirige dinámicamente las llamadas directamente al endpoint oficial de Google Gemini, garantizando que el asistente siempre responda con fiabilidad.